Distributed Key-Value Store

Consistent hashing, replication, TTL, node routing, simulated failure

Project Type	Best For	Deliverables
Backend / Distributed Systems Demo	Amazon SDE I resume + interview explanation	Runnable code, demo script, tests, PDFs

1. Problem Statement

Distributed databases need to route keys across nodes, replicate data, expire stale records, and continue operating when nodes fail.

2. Architecture Flow

Client -> KV API -> Consistent Hash Ring -> Node Router -> Replica Selection -> Storage Nodes -> TTL Cleaner

3. Core Features

- GET, PUT and DELETE key-value APIs
- Consistent hashing with virtual nodes
- Replication factor of two across healthy nodes
- TTL expiration for temporary keys
- Simulated node down/up operations
- Tests for routing, replication and reads

4. API Surface

Endpoint	Purpose
PUT /kv/{key}	Write key with optional TTL
GET /kv/{key}	Read from routed replica
DELETE /kv/{key}	Delete from replicas
GET /nodes	Inspect node status and keys
POST /nodes/{node}/down	Simulate node failure

5. How to Run the Demo

Run these commands from the project folder:

```
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
pytest -q
```

6. Demo Talking Points

This project is useful for SDE interviews because I can explain hashing, replication, routing, failure handling, and why distributed storage is harder than a single database.

7. Resume Bullet

> Engineered a distributed key-value store with consistent hashing, TTL support, replication, simulated node failure, key routing, and automated tests.

8. What to Say in Interview

- Why I built it: to practice backend workflows that need correctness, reliability and scalability.
- Main challenge: keeping state transitions correct when retries or failures happen.
- What I learned: data structures, APIs, testing, failure handling, and clean modular design.
- Future scale-up: replace in-memory components with Redis/Kafka/SQS, add auth, metrics, tracing and cloud deployment.

9. Demo Output Snapshot

PUT /kv/cart:101


```
{
"key": "cart:101",
"stored_on": [
"node-3",
"node-4"
],
"replication_factor": 2
}
```


GET /kv/cart:101


```
{
"key": "cart:101",
"value": "keyboard",
"served_by": "node-3",
"replicas": [
"node-3",
"node-4"
]
}
```


GET /nodes

```
{
"nodes": {
"node-1": {
"status": "UP",
"keys": []
},
"node-2": {
"status": "UP",
"keys": []
},
"node-3": {
"status": "UP",
"keys": [
"cart:101"
]
},
"node-4": {
"status": "UP",
"keys": [
"cart:101"
]
}
},
"ring_virtual_nodes": 128
}
```